

Embeddings y Bases de Datos Vectoriales

Módulo 7 · Sistemas RAG y Gestión del Conocimiento

Voyage AI vs OpenAI embeddings, Pinecone vs Qdrant vs Weaviate, búsqueda híbrida y proceso de selección con MTEB.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Embeddings y Bases de Datos Vectoriales

Si RAG es el sistema, las bases de datos vectoriales son su corazón. Son las que hacen posible buscar entre millones de documentos en milisegundos por similitud semántica, algo imposible con bases de datos tradicionales. La elección del modelo de embedding y de la base vectorial determina la calidad de la recuperación, la latencia, el coste de escalado, y la viabilidad técnica del sistema RAG en producción.

En diciembre de 2025, el mercado de bases de datos vectoriales se consolida rápidamente. Voyage AI lidera los benchmarks MTEB superando a OpenAI text-embedding-3-large en un 9.74% y a Cohere embed-v3 en un 20.71%, con ventanas de contexto de 32K tokens (vs 8K de OpenAI) a menor coste. Las bases vectoriales Pinecone, Weaviate, Milvus y Qdrant son las cuatro opciones principales para producción.

Modelos de embedding para RAG

Los modelos líderes en 2025

Voyage AI voyage-3-large: 1024 dimensiones, contexto 32K tokens, \$0.06/M tokens. Lidera MTEB. Ideal para dominios especializados (legal, médico, financiero). OpenAI text-embedding-3-large: 3072 dimensiones, contexto 8K tokens, \$0.13/M tokens. Más conocido, mayor soporte de ecosistema. Cohere Embed v3: multilingüe, buena opción para corpus en múltiples idiomas. Voyage AI embed-multilingual-v3: mejor que Cohere para corpus multilingüe con texto técnico. Open source (Hugging Face): BAAI/bge-large-en-v1.5 (gratuito, buena calidad) y sentence-transformers/all-MiniLM-L6-v2 (muy rápido, menor calidad) para casos donde el coste es prioritario.

Las cuatro bases vectoriales principales

Pinecone: SaaS managed, la más fácil de empezar, sin gestión de infraestructura, buen soporte para filtros de metadatos, escalado automático. Precio basado en uso + índices. Ideal para equipos sin experiencia en infraestructura de datos. Weaviate: open source + cloud, buen balance entre funcionalidades y facilidad. Soporta búsqueda híbrida (vector + keyword) nativamente, tiene módulos para generación directa. Milvus: open source, diseñado para escala masiva (decenas de miles de millones de vectores). Mayor complejidad de operación. Qdrant: open source (Rust), mayor rendimiento en throughput, muy fácil de auto-hostear con Docker, soporte nativo para payload filtering y búsqueda híbrida. Crecimiento rápido en 2024-2025.

FAISS: la librería para búsqueda vectorial en memoria

FAISS (Facebook AI Similarity Search) es la librería de referencia para búsqueda vectorial cuando no se necesita una base de datos completa con persistencia. Permite indexar millones de vectores en RAM y buscar los K más similares en milisegundos. Tiene múltiples tipos de índice: IndexFlatL2 (búsqueda exacta, $O(n)$), IndexIVFFlat (búsqueda aproximada con particiones, más rápido), IndexIVFPQ (cuantización de producto, menor memoria a cambio de

algo de precisión). FAISS es la base sobre la que están construidas ChromaDB y otras bases vectoriales de prototipado.

SECCIÓN 3 · CÓMO FUNCIONA

Indexación y búsqueda en bases vectoriales

El proceso de indexación

Para indexar una colección de documentos en una base vectorial: genera un embedding de cada chunk usando el modelo elegido (en batch para eficiencia), crea un registro en la base vectorial con el vector de embedding, el texto del chunk, y los metadatos relevantes (nombre del documento, fecha, categoría, permisos de acceso, posición en el documento). El tiempo de indexación depende del volumen: con text-embedding-3-large a \$0.13/M tokens, 1M de documentos de 1000 tokens cuesta ~\$130 en embeddings y toma horas en modo batch.

Tipos de búsqueda: densa, dispersa e híbrida

Búsqueda densa (vector search): convierte la consulta en embedding y busca los vectores más cercanos por similitud coseno. Captura similitud semántica pero puede fallar en búsquedas de términos exactos o nombres propios. Búsqueda dispersa (sparse/keyword): BM25 o TF-IDF sobre los tokens del texto. Excelente para términos exactos, acrónimos, nombres propios. No captura similitud semántica. Búsqueda híbrida: combina ambas con fusión de rankings (RRF, Reciprocal Rank Fusion, o score lineal). Un estudio de IBM Research de abril 2024 (BlendedRAG) demostró que la búsqueda híbrida produce mejores resultados que cualquier método individual. Semantic chunking mejora el recall hasta un 9% sobre el chunking de tamaño fijo.

Filtrado por metadatos

Las bases vectoriales modernas soportan filtrado por metadatos: buscar los K vectores más similares entre los documentos que cumplen ciertos criterios de metadatos (fecha > 2024, categoría = "RRHH", permisos = usuario_actual). Este filtrado pre o post-retrieval es fundamental para sistemas multi-tenant donde cada usuario solo puede ver sus propios documentos, o para sistemas que necesitan limitar la búsqueda a un subconjunto de la base de conocimiento.

SECCIÓN 4 · EJEMPLOS REALES

- Pinecone en sistema de soporte al cliente (Zendesk): índice de 5M de tickets históricos y artículos de knowledge base. Recupera los 5 documentos más relevantes en <50ms por consulta. La búsqueda semántica permite encontrar tickets similares aunque usen vocabulario diferente al del usuario actual.
- Qdrant en búsqueda de código (empresas tecnológicas): índice de repositorios de código con embeddings de función + descripción. Los desarrolladores buscan funciones similares en el codebase usando lenguaje natural. Qdrant auto-hosteado en Docker reduce los costes de SaaS para volúmenes altos de búsqueda interna.
- Weaviate en plataforma de investigación científica (Elsevier): índice de millones de papers científicos con búsqueda híbrida (semántica + keywords de términos técnicos). Los

investigadores encuentran papers relevantes por concepto aunque no conozcan los términos exactos del campo.

- Milvus en sistema de recomendación de e-commerce (Alibaba): índice de cientos de millones de productos con embeddings multimodales (texto + imagen). La búsqueda por similitud de producto permite recomendaciones más relevantes que los sistemas de filtros colaborativos clásicos.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: No actualizar los embeddings cuando cambian los documentos. Si un documento fuente se actualiza pero su embedding no, el sistema recuperará el chunk obsoleto y generará respuestas basadas en información desactualizada. Implementa un pipeline de actualización incremental que regenere embeddings cuando los documentos cambian. Versiona los documentos y sus embeddings.

Error 2: Elegir la dimensionalidad del embedding por coste de almacenamiento sin evaluar el impacto en la calidad. Los embeddings de mayor dimensionalidad (3072 vs 1024) capturan matices más finos pero cuestan más en almacenamiento y búsqueda. La reducción de dimensionalidad de OpenAI text-embedding-3 a 256 dimensiones puede reducir costes 12x con solo 1-2% de pérdida de calidad en tareas generales, pero puede ser significativa en dominios especializados.

Error 3: Usar el modelo de embedding equivocado para el tipo de consulta. Los modelos de embedding asimétricos (query-document) están diseñados para que la consulta del usuario y los documentos tengan formatos distintos (E5, BGE-M3). Los modelos simétricos (all-MiniLM) asumen que query y documentos tienen el mismo formato. Para RAG, los modelos asimétricos suelen dar mejores resultados porque las consultas de usuario son muy distintas en forma a los fragmentos de documentos.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Proceso de selección de modelo de embedding: consulta el MTEB Leaderboard (huggingface.co/spaces/mteb/leaderboard) para ver los mejores modelos por categoría de tarea y por idioma. Selecciona 2-3 candidatos. Construye un mini-corpus de 200-500 chunks de tu dominio y 50-100 pares (pregunta, documento relevante). Evalúa el recall@5 y recall@10 de cada modelo en ese mini-corpus. El modelo con mejor recall en tu dominio es el que debes usar.

Para empezar con Qdrant en local: `docker run -p 6333:6333 qdrant/qdrant`. El cliente Python es `qdrant-client`. Para crear una colección con búsqueda HNSW: `client.create_collection(collection_name="docs", vectors_config=VectorParams(size=1024, distance=Distance.COSINE))`. Para upsert vectores con metadatos: `client.upsert(collection_name="docs", points=[PointStruct(id=i, vector=embedding, payload={"text": chunk_text, "source": doc_name})])`. Para buscar: `client.search(collection_name="docs", query_vector=query_embedding, limit=5)`.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M2-T6 y M2-T7 (Embeddings y espacios vectoriales): los fundamentos matemáticos de los embeddings y la similitud coseno son la base de la búsqueda vectorial.
- M7-T3 (Indexación y chunking): la base vectorial almacena los chunks; este tema cubre cómo dividir los documentos en chunks.
- M7-T4 (Búsqueda semántica): la búsqueda vectorial es el corazón de la búsqueda semántica.

SECCIÓN 8 · RESUMEN EJECUTIVO

Los embeddings convierten texto en vectores que capturan significado semántico. Modelos líderes en 2025: Voyage AI voyage-3-large (mejor MTEB, 32K contexto), OpenAI text-embedding-3-large (más extendido), BGE/E5 (open source de calidad). Bases vectoriales principales: Pinecone (SaaS, más fácil), Weaviate (features completos), Milvus (escala masiva), Qdrant (rendimiento + facilidad de auto-hosting). Búsqueda híbrida (semántica + keyword) supera a cualquier método individual. El semantic chunking mejora el recall un 9% sobre chunking de tamaño fijo. El proceso de selección correcto: evaluar con mini-corpus propio en MTEB antes de decidir.